



UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

Sistemas de Computación

Trabajo Práctico 4

Módulos de kernel

Integrantes

Gonzalez Leahy María Clara

Nuñez Anna

Vigazzi Ignacio Joaquin

1. ¿Qué diferencias se pueden observar entre los dos modinfo ?

```
vm@virtualm:~/Projects/kernel-modules/part1/module$ modinfo mimodulo.ko
filename:       /home/vm/Projects/kernel-modules/part1/module/mimodulo.ko
author:        Catedra de SdeC
description:    Primer modulo ejemplo
license:       GPL
srcversion:    C6390D617B2101FB1B600A9
depends:
  name:        mimodulo
  retpoline:   Y
  vermagic:    6.14.0-37-generic SMP preempt mod unload modversions
vm@virtualm:~/Projects/kernel-modules/part1/module$ modinfo /lib/modules/$(uname -r)/kernel/fs/fat/msdos.ko.zst
filename:       /lib/modules/6.14.0-37-generic/kernel/fs/fat/msdos.ko.zst
description:    MS-DOS filesystem support
author:        Werner Almesberger
license:       GPL
alias:         fs-msdos
srcversion:    2FD7471446589ECF4390491
depends:
  intree:      Y
  name:        msdos
  retpoline:   Y
  vermagic:    6.14.0-37-generic SMP preempt mod unload modversions
sig_id:        PKCS#7
signer:        Build time autogenerated kernel key
sig_key:       1E:D4:CC:DF:F3:2A:D3:7C:B5:68:B6:72:F3:90:D9:67:06:F8:7D:EB
sig_hashalgo:  sha512
signature:     96:ED:62:33:F8:94:D1:7C:DF:CE:54:BB:56:1E:7C:3E:88:76:3A:63:
                2E:E6:9D:37:86:8F:AF:B4:9A:AF:93:F8:62:75:38:61:12:43:14:89:
                2E:F1:80:6D:1E:00:31:9C:35:F7:80:ED:43:53:74:99:20:49:13:49:
                54:5A:4B:9B:23:52:50:10:1E:A7:CF:04:D6:D9:5F:4F:C0:A3:C0:74:
                7D:82:5C:C0:58:3E:86:16:38:0A:5F:99:CC:FE:41:1D:4F:D5:0E:8D:
                11:FC:A7:EB:C3:44:1F:25:86:A9:E1:E7:9E:D9:6D:85:2A:C3:70:14:
                C4:2B:E0:3A:B7:F9:D8:6A:46:C4:D4:9D:03:D3:81:0C:70:9C:24:02:
                54:50:3B:35:8A:55:05:06:0B:62:75:00:37:53:50:05:70:11:65:03:
```

Se puede ver cómo, además de cambiar los campos de datos propios de cada módulo como nombre y autor, podemos encontrar en el modinfo del módulo del sistema de archivos fat32 información correspondiente a la firma digital con la que se validó el mismo. El firmado digital se utiliza como medida de seguridad ya que la carga de código malicioso en el kernel compromete al sistema de manera crítica, por lo que puede establecerse que solo se pueda cargar módulos cuyo origen está verificado y no fueron modificados.

2. ¿Qué divers/modulos estan cargados en sus propias pc?

Se puede obtener una lista de los módulos cargados con el comando lsmod, (https://github.com/Igna124/Tiny_admins/blob/TP4/tp4/outputAnny.txt https://github.com/Igna124/Tiny_admins/blob/TP4/tp4/output.txt).

Algunos de los módulos:

- Módulos de audio (snd_intel8x0, snd_ac97_codec, snd_pcm): controladores de la placa de sonido.
- Módulos de máquina virtual (vboxguest, vboxvideo): permiten la integración de Ubuntu con el Windows anfitrión.
- Módulos de seguridad y criptografía (aesni_intel, ghash_clmulni_intel): permiten al SO usar las instrucciones criptográficas del procesador Intel.

Al comparar la salida de lsmod entre dos equipos diferentes se puede ver como ambas difieren en numerosos módulos debido a los diferentes periféricos, hardware y controladores utilizados en cada sistema. Esto evidencia, por ejemplo, que una se trata de una computadora física mientras que la otra se está ejecutando en una máquina virtual debido a los múltiples controladores de virtualización. Además, un sistema utiliza un chipset de audio HDA moderno mientras que el otro utiliza AC97 más antiguo.

3. ¿Cuáles no están cargados pero están disponibles? Qué pasa cuando el driver de un dispositivo no está disponible.

El comando lsmod solo muestra los módulos cargados. Para listar los módulos disponibles se debe revisar el contenido de la carpeta /lib/modules/ donde se encuentran todos los drivers que posee el sistema (ver module_list.txt).

Si el driver de un dispositivo no está disponible, el sistema no tendrá las instrucciones necesarias para controlarlo. Esto puede resultar en que el dispositivo tenga funcionalidad reducida, no sea reconocido o incluso genere errores del sistema.

4. Correr hwinfo en una pc real con hw real y agregar la url de la información de hw en el reporte.

https://github.com/lgna124/Tiny_admins/blob/TP4/hwinfo.txt

5. ¿Qué diferencia existe entre un módulo y un programa ?

La principal diferencia entre un módulo y un programa es donde se ejecuta y los privilegios que tiene. Un programa es un archivo que se ejecuta aisladamente en un espacio de usuario, con privilegios restringidos de manera lineal, sin que esto genere riesgos al ocurrir una falla. Y un módulo es un segmento de código capaz de cargarse y descargarse de forma dinámica directamente en el espacio del kernel según necesidad. Esto permite añadir funcionalidades al núcleo sin tener que reconstruir la imagen del kernel ni reiniciar el sistema. Además al estar en el espacio del kernel se tiene privilegios máximos y el riesgo al ocurrir fallas es muy alto.

6. ¿Cómo puede ver una lista de las llamadas al sistema que realiza un simple helloworld en c?

Para ver las llamadas al sistema que realiza un programa se utiliza el comando strace. Si se aplica a un helloworld escrito en c se obtiene la siguiente salida:

[illegible]

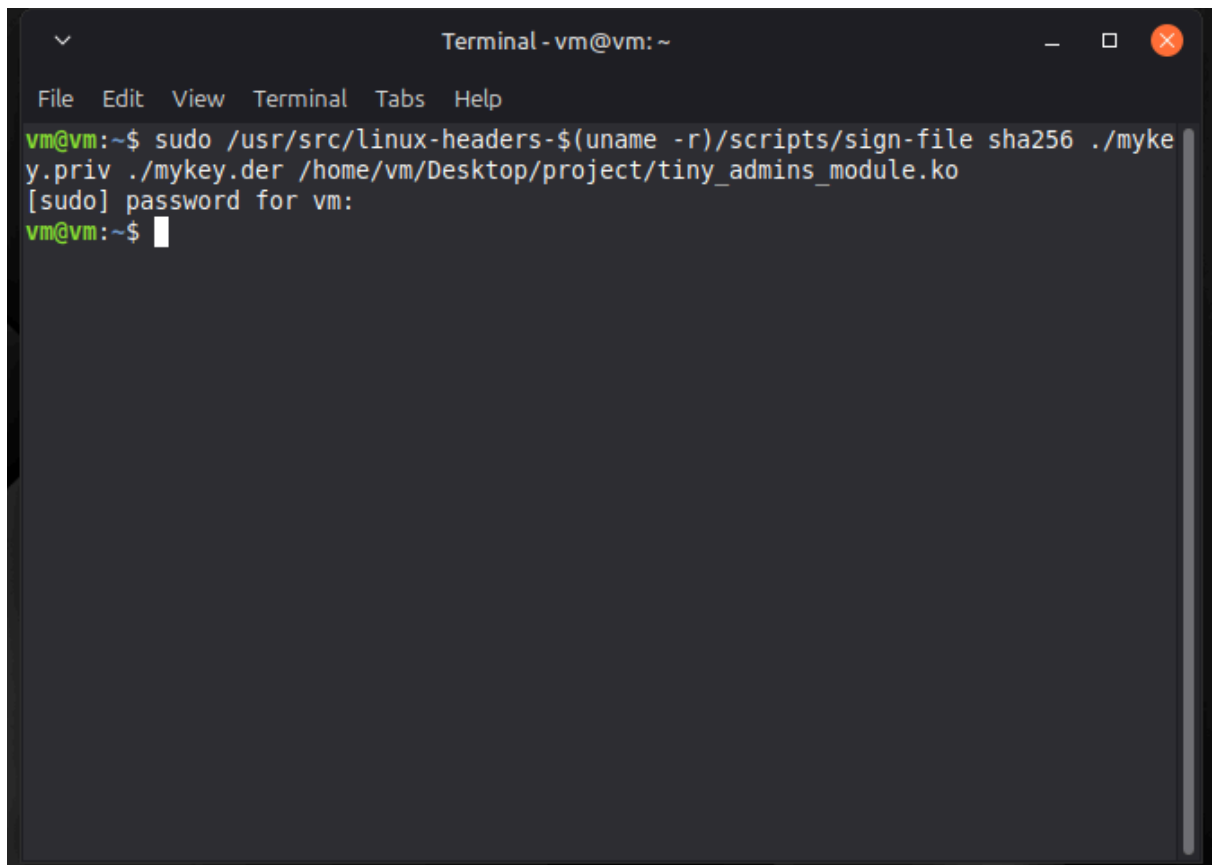
Donde se pueden ver por ejemplo la llamada a la ejecución del programa con `execve`, lectura y escritura de descriptores de archivos y mapeos de memoria con `mmap`

7. ¿Qué es un segmentation fault? ¿Cómo lo maneja el kernel y como lo hace un programa?

Un segmentation fault es una señal de error que genera el SO cuando un programa intenta acceder o modificar a una dirección de memoria que no le pertenece/no tiene acceso. El Kernel en respuesta a este intento de acceso toma el control, identifica el proceso que intenta acceder y le envía una señal de violación de segmento (SIGSEGV). Se elimina el proceso para evitar corromper la memoria, se imprime error por la terminal y se cierra el programa. El programa por su parte se interrumpe y muere en esa línea de código. Si se tiene configurada un manejador de señales, el programa captura la señal antes del cierre y salva datos de emergencia o escribe en un registro los detalles del fallo.

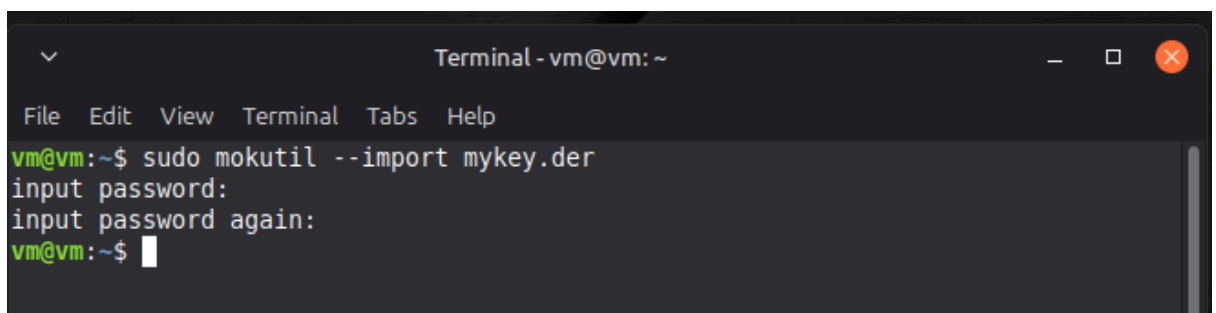
8. ¿Se animan a intentar firmar un módulo de kernel ? y documentar el proceso ?
9. Agregar evidencia de la compilación, carga y descarga de su propio módulo imprimiendo el nombre del equipo en los registros del kernel.

Para firmar un módulo primero se debe generar la clave y el certificado que se utilizará para verificar la firma con el comando *openssl req*.

A terminal window titled "Terminal - vm@vm: ~" with a menu bar (File, Edit, View, Terminal, Tabs, Help). The prompt is "vm@vm:~\$". The user enters the command: `sudo /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ./mykey.priv ./mykey.der /home/vm/Desktop/project/tiny_admins_module.ko`. The terminal shows the command being executed, followed by a password prompt: `[sudo] password for vm:`. The prompt returns to `vm@vm:~$` with a cursor.

```
vm@vm:~$ sudo /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ./mykey.priv ./mykey.der /home/vm/Desktop/project/tiny_admins_module.ko
[sudo] password for vm:
vm@vm:~$
```

Para que el sistema reconozca el certificado como válido se lo debe agregar a la lista de certificados de secure boot manualmente

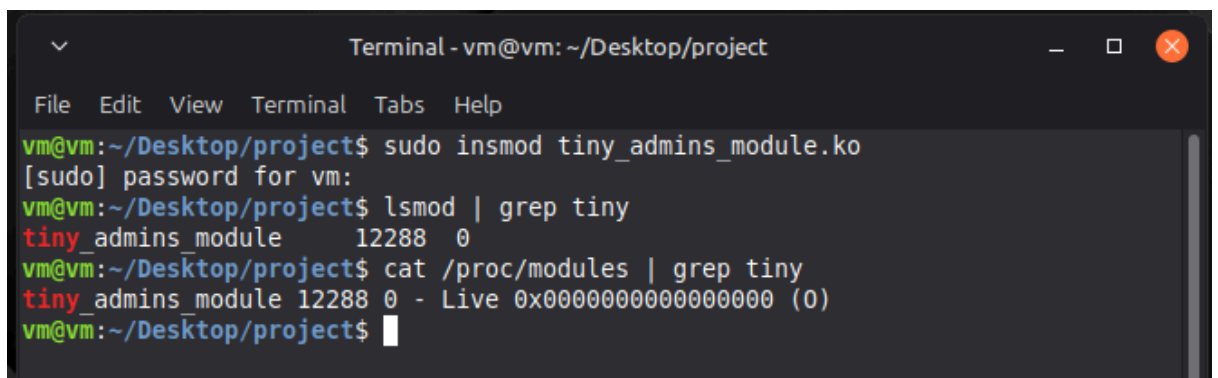
A terminal window titled "Terminal - vm@vm: ~" with a menu bar (File, Edit, View, Terminal, Tabs, Help). The prompt is "vm@vm:~\$". The user enters the command: `sudo mokutil --import mykey.der`. The terminal shows the command being executed, followed by two password prompts: `input password:` and `input password again:`. The prompt returns to `vm@vm:~$` with a cursor.

```
vm@vm:~$ sudo mokutil --import mykey.der
input password:
input password again:
vm@vm:~$
```

Al reiniciar el sistema, se muestra la información del certificado y se guarda en la memoria de UEFI.



Una vez registrado el certificado, el sistema reconoce los modulos firmados con la clave como seguros y pueden ser cargados en el kernel con secure boot activado.



```
Terminal - vm@vm: ~/Desktop/project
File Edit View Terminal Tabs Help
ation="profile_load" profile="unconfined" name="ch-run" pid=450 comm="apparmor_p
arser"
[ 7.961368] audit: type=1400 audit(1779343474.403:13): apparmor="STATUS" oper
ation="profile_load" profile="unconfined" name="ch-checkns" pid=449 comm="apparm
or_parser"
[ 8.168230] RAPL PMU: API unit is 2^-32 Joules, 0 fixed counters, 10737418240
ms ovfl timer
[ 8.241624] i801_smbus 0000:00:1f.3: Enabling SMBus device
[ 8.241948] i801_smbus 0000:00:1f.3: SMBus using PCI interrupt
[ 8.251928] i2c i2c-0: Memory type 0x07 not supported yet, not instantiating
SPD
[ 10.094741] NET: Registered PF_QIPCRTR protocol family
[ 12.167987] e1000e 0000:00:02.0 enp0s2: NIC Link is Up 1000 Mbps Full Duplex,
Flow Control: Rx/Tx
[ 12.216235] Lockdown: Xorg: raw io port access is restricted; see man kernel_
lockdown.7
[ 14.413457] systemd-journald[296]: /var/log/journal/7c2e9077d4234ac1a205e8f83
19307d6/user-1000.journal: Journal file uses a different sequence number ID, rot
ating.
[ 57.004225] tiny_admins_module: loading out-of-tree module taints kernel.
[ 57.005094] Este equipo ahora es propiedad de Tiny Admins.
vm@vm:~/Desktop/project$ mokutil --sb-state
SecureBoot enabled
vm@vm:~/Desktop/project$
```

```
Terminal - vm@vm: ~/Desktop/project
File Edit View Terminal Tabs Help
arser"
[ 7.961368] audit: type=1400 audit(1779343474.403:13): apparmor="STATUS" oper
ation="profile_load" profile="unconfined" name="ch-checkns" pid=449 comm="apparm
or_parser"
[ 8.168230] RAPL PMU: API unit is 2^-32 Joules, 0 fixed counters, 10737418240
ms ovfl timer
[ 8.241624] i801_smbus 0000:00:1f.3: Enabling SMBus device
[ 8.241948] i801_smbus 0000:00:1f.3: SMBus using PCI interrupt
[ 8.251928] i2c i2c-0: Memory type 0x07 not supported yet, not instantiating
SPD
[ 10.094741] NET: Registered PF_QIPCRTR protocol family
[ 12.167987] e1000e 0000:00:02.0 enp0s2: NIC Link is Up 1000 Mbps Full Duplex,
Flow Control: Rx/Tx
[ 12.216235] Lockdown: Xorg: raw io port access is restricted; see man kernel_
lockdown.7
[ 14.413457] systemd-journald[296]: /var/log/journal/7c2e9077d4234ac1a205e8f83
19307d6/user-1000.journal: Journal file uses a different sequence number ID, rot
ating.
[ 57.004225] tiny_admins_module: loading out-of-tree module taints kernel.
[ 57.005094] Este equipo ahora es propiedad de Tiny Admins.
[ 285.562728] Tiny Admins se retira de su dispositivo, tenga buen dia.
vm@vm:~/Desktop/project$ mokutil --sb-state
SecureBoot enabled
vm@vm:~/Desktop/project$
```


10. ¿Que pasa si mi compañero con secure boot habilitado intenta cargar un módulo firmado por mi?

11. Dada la siguiente nota

<https://arstechnica.com/security/2024/08/a-patch-microsoft-spent-2-years-preparing-is-making-a-mess-for-some-linux-users/>

a. ¿Cuál fue la consecuencia principal del parche de Microsoft sobre GRUB en sistemas con arranque dual (Linux y Windows)?

El parche de Microsoft provocó que los sistemas de arranque dual no pudieran usar el dispositivo con Linux como SO si tenían el Secure Boot activado.

b. ¿Qué implicancia tiene desactivar Secure Boot como solución al problema descrito en el artículo?

Desactivar el Secure Boot deja vulnerable frente a ataques a nivel de firmware.

c. ¿Cuál es el propósito principal del Secure Boot en el proceso de arranque de un sistema?

El propósito principal del Secure Boot en el proceso de arranque de un sistema es proteger el firmware de ataques y software malicioso durante el proceso de arranque del SO.